STR: Hybrid Tensor Re-Generation to Break Memory Wall for DNN Training

Zan Zong, Li Lin, Leilei Lin, Lijie Wen, and Yu Sun

***Abstract*—With the growth of the depth of neural networks and the scale of data, the difficulty of network training also increases. When the GPU memory is insufficient, it is challenging to train deeper models. Recent research uses tensor swapping and recomputation techniques in a combined manner to optimize memory usage. However, complex dependencies and enormous scales of the DNN graph limit the improvement of single GPU memory optimization. Improper swap and recomputation decisions even bring negative effects on training performance. In this article, we propose a novel hybrid tensor re-generation strategy, called STR, which combines swap and recomputation techniques to find the optimal execution plan for the DNN training when the memory is limited. We formalize our memory optimization problem with constraints that describe the dependency of the operator calculation and the bandwidth usage of the swap. A *host checkpoint* mechanism is designed to make full use of the swapped tensors, which reduces the cost of the recomputation. We also present a *recursive source tracing* algorithm to improve the optimization efficiency by constraint relaxation with a performance bound. To optimize large models, we further introduce an approximation method based on a weighted graph coarsening. We implement a prototype of STR as a plugin on TensorFlow and evaluated based on 5 popular DNN models. The experimental result shows that the approximate solution of STR improves the training throughput of ResNet series of models by up to 28.1% compared to the state-of-the-art hybrid optimization strategy.**

***Index Terms*—DNN training, offload memory, recomputation, rematerialization, swap.**

I. Introduction

THE success of deep learning has drawn much attention from both the academic and industrial fields in recent years. According to the published trace of Philly [1], we found that there are 87% of 112018 DNN training jobs requesting a single GPU for training. However, efficiently training models within a single GPU becomes more and more challenging because the state-of-the-art deep learning models continue to grow. Under a specific GPU card, the training will meet an upper bound when the peak memory per iteration reaches the memory budget. Insufficient GPU memory becomes one of the most prominent challenges for DNN training [2], [3], [4]. In this scenario, memory optimization is needed to reduce the memory footprint to facilitate the training.

According to the training mechanism of DNN models, there is a lot of structural data resident in GPU memory such as parameters, activations and gradients. In this paper, we focus on how to optimize the memory usage of activations because this part of the memory footprint consumes most of the GPU memory during training [5], [6]. When a tensor (i.e., activation) is generated by an operation in the forward propagation, it will not be dropped until accessed in the backward propagation to calculate the gradient, which prevents the training of larger models. To reduce memory usage, many approaches have been studied to investigate how to deallocate tensors that are not immediately accessed and re-generate them back when needed. *Swap* and *recomputation* are two techniques that have been used to tackle such a problem.

The *swap* technique utilizes the relatively large space of CPU memory, which is up to hundreds of gigabytes in modern GPU servers. For example, Chen et al. [7] presented an approach that utilizes host memory as a bigger memory pool with "swap-out/in" operations, which overcomes the limitation of the GPU memory. The *recomputation* means that a tensor will be freed when the memory is insufficient and be re-generated by computing from its sources when it is used again. However, the recomputation needs extra computing costs and immoderate recomputations will lead to a degradation of the training performance. Checkmate [5] tries to find an optimal recomputation strategy to minimize the extra recomputation cost and sustain the DNN training. To consider both swap and recomputation, Peng et al. [8] proposed a method that uses both swap and recomputation techniques greedily. However, due to the complex dependency of tensors and the rule-based selection of tensors to be swapped, sophisticated greedy strategies are difficult to achieve optimality. We hope to use a hybrid approach to fully utilize the bandwidth between GPU memory and host memory (e.g., PCIe 3.0 $\times$ 16) to reduce the recomputation overhead, which improves the training performance for the case that GPU memory is insufficient.

Manuscript received 9 June 2022; revised 7 February 2023; accepted 24 March 2023. Date of publication 10 April 2023; date of current version 5 July 2023. This work was supported by the National Key Research and Development Program of China under Grant 2019YFB1704003, in part by the National Nature Science Foundation of China under Grant 62021002, in part by the R&D Program of Beijing Municipal Education Commission under Grant KM202310028003, in part by the Natural Science Foundation of Tianjin under Grant 22JCQNJC01520, and in part by the Tsinghua BNRist and Beijing Key Laboratory of Industrial Big Data System and Application. Recommended for acceptance by M. Si. *(Corresponding author: Lijie Wen.)*

Zan Zong is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: zongz@tsinghua.edu.cn).

Li Lin is with the School of Computer Science and Engineering, Southeast University, Nanjing, Jiangsu 211189, China (e-mail: linli321@seu.edu.cn).

Leilei Lin is with the School of Management, Capital Normal University, Beijing 100048, China (e-mail: leilei_lin@cnu.edu.cn).

Lijie Wen is with the School of Software, Tsinghua University, Beijing 100084, China (e-mail: wenlj@tsinghua.edu.cn).

Yu Sun is with the College of Computer Science, Nankai University, Tianjin 300350, China (e-mail: sunyu@nankai.edu.cn).

Digital Object Identifier 10.1109/TPDS.2023.3266110

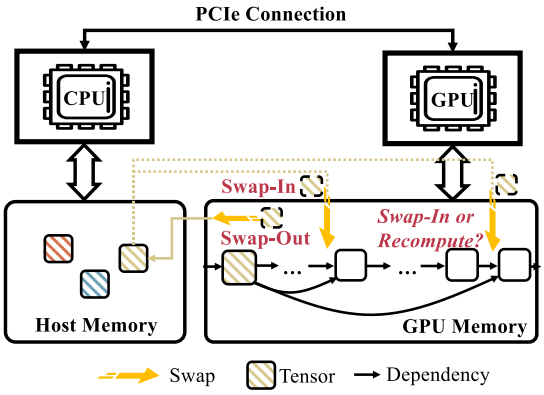


Fig. 1. We consider a common GPU server where the GPU and the CPU are connected via PCIe. Host checkpoints may be swapped into GPU multiple times.

In the context of the recomputation technique, the optimizer selects only a few tensors to reside in GPU memory and recomputes other tensors when needed. The tensor remaining in GPU memory is known as a checkpoint [9], [10]. Checkpoints can efficiently reduce the recomputation overhead when they are the parents of the recomputed tensors. Borrowing this concept, we introduce *host checkpoint*, which refers to the tensor swapped out to the host memory that can be accessed as the checkpoint on demand. As illustrated in Fig. 1, for a training iteration, once a tensor is swapped out and becomes a host checkpoint, it can be swapped in once or multiple times as long as the bandwidth resources are not exhausted. We hope the recomputation covers only cases where host checkpoints cannot be swapped in time, which achieves the minimal extra recomputation cost.

There are three challenges in achieving our targets. First, it is hard to accurately determine when to trigger the swap-in and the swap-out operation. Swapping out late cannot alleviate the lack of memory, and swapping in late will suspend the calculation of downstream operators. Second, tensors can be re-generated in the different swap and recomputation methods. Due to the various DNN architectures, a tensor can be the source of multiple operators. If the tensor is swapped out to host memory, we prefer to schedule multiple times of swap-in. When it is impossible to swap in due to limited bandwidth resources, we need to consider re-generating the tensor through the recomputation. Third, finding the exact solution is time-consuming for models with deep layers. It has been proved that the optimal swapping problem is NP-complete [6]. Considering both the swap and the recomputation will further increase the difficulty and lead to unacceptable time consumption for large DNN models.

To tackle these challenges, we introduce STR, which can be used to find the solution at low computational costs using a **S**wap-dominated **T**ensor **R**e-generation method. STR considers the competition of bandwidth resources through the *inter-tensor* and the *intra-tensor* constraints, providing a solution that tends to use the swap as much as possible to fully utilize the PCIe bandwidth. STR also provides approximation approaches based on constraint relaxation and graph coarsening to accelerate the optimization for large DNN models.

In summary, we make the following contributions to this work:

- We formalize the GPU memory optimization problem as a mixed integer linear programming problem. The optimal solution with minimal recomputation costs will be found through a series of constraints on the use of swap and recomputation, thereby obtaining the characteristics of the *host checkpoint*.
- To accelerate the solving efficiency, we introduce a *recursive source tracing* algorithm based on the constraint relaxation. We also prove that the training performance does not deteriorate after relaxation in memory-insufficient situations.
- We introduce a graph coarsening approximation based on constraint relaxation, which provides the capability of optimizing large DNN models within acceptable time costs.
- We implement a prototype of STR as a plugin of TensorFlow. Five commonly used neural networks are chosen for evaluation. The experimental result shows that for the memory-limited scenario, compared with the state-of-the-art hybrid optimization approach Capuchin [8], recomputation-based approach Checkmate [5] and swap-based approach DYNPROG [6], STR improves throughput by 22.7%, 22.2% and 16.1%, respectively. The graph coarsening also outperforms Capuchin by up to 30.6% for training throughput.

Our preliminary work has been presented in [11], which introduces a series of constraints but lacks the capability of optimizing large DNN models. Compared to this work, we split long stages into mini-stages to provide more opportunities for swapping, which is detailed in Section III-C. We ameliorate the constraint relaxation algorithm and provide a theoretical performance bound in Section IV. Then we further design a coarsening-solving-recovering procedure to optimize large DNN models, which is detailed in Section V. Evaluations on large models such as ResNet152 and Transformer are also supplied accordingly.

In the following sections, we introduce the background of GPU memory optimization and our intuitions in Section II. Section III presents the problem definition and optimization strategy. Section IV provides a constraint relaxation algorithm with a performance guarantee. An approximation is introduced in Section V to further speed up the optimization. The implementation of STR is detailed in Section VI. We evaluate STR in Section VII and summarize related work in Section VIII. Finally, we conclude this work in Section IX.

II. Background and Motivation

In this section, we will investigate the existing studies and show why there are still some opportunities to further improve the training efficiency by a hybrid approach.

A. Swap Technique

During the DNN training, feature maps (i.e., tensors) must be stored in GPU memory after the forward pass until the gradient has been calculated in the backward pass. Feature maps are the main ingredient of GPU memory usage [5], [7]. To alleviate the pressure of memory, some studies evict the tensors generated in

TABLE I
EXISTING RESEARCH STUDIES WITH DIFFERENT SOLUTIONS

Studies	Highlights	Techniques	Exact Solution
Chen et al. [12]	The memory consumption can be reduced to $O(logN)$ with memory optimized gradient graph construction.	Recomputation	✗
DTR [13]	A greedy online algorithm that closely matches the performance of static checkpointing strategy of Chen et al. [12].	Recomputation	✗
Meng et al. [7]	Utilizing host memory as a bigger memory pool to save tensors.	Swap	✗
vDNN [14]	Employing two CUDA streams to overlap normal computation and memory offload.	Swap	✗
vDNN++ [15]	Improving vDNN [14] with asynchronous transfer and heuristics to address memory fragmentation.	Swap	✗
SwapAdvisor [16]	Optimizing GPU memory along 3 dimension: operator scheduling, memory allocation and swap decisions.	Swap	✗
TFLMS [17]	An operator-based swapping mechanism implementation that heuristically find control dependency operations.	Swap	✗
SuperNeurons [18]	Breaking GPU memory wall with liveness analysis, unified tensor pool and cost-aware recomputation.	Hybrid	✗
Capuchin [8]	Measuring tensor re-generation costs and determining memory optimization heuristically.	Hybrid	✗
Kusumoto et al. [19]	Formalizing recomputation problem with graph theory and solving by dynamic programming.	Recomputation	✓
Checkmate [5]	Formalizing recomputation problem as a mixed integer linear program and solving it with numerical optimizers.	Recomputation	✓
Beaumont et al. [6]	Proving that the DNN memory offload problem is NP-complete and presenting two heuristic solutions.	Swap	✓
STR (ours)	Solving the hybrid optimization to get the exact solution and two approximated solutions.	Hybrid	✓

the forward pass of training from the GPU memory to the host memory and copy the tensors back to the GPU when needed. This mechanism is called *swap* or *offload*. *Swap-out* and *swap-in* are two important operations to achieve the GPU-to-host and the host-to-GPU tensor transmission respectively. For example, SuperNeurons [18] chooses to swap the output of convolution layers. SwapAdvisor [16] tends to swap out tensors not needed for the longest time in the future.

B. Recomputation and Checkpoint

To break the limitation of GPU memory, many studies try to drop the current unused tensors and recompute them back when they are required again. It is important to know which tensors are suitable to be dropped and recomputed. Chen et al. [12] proposed to drop the results of low-cost operations and recover them from extra forward recomputation when needed. To reduce the extra forward recomputation costs, some tensors reside in GPU memory after the forward pass of training, and serve as computation sources for subsequent operations, which is known as the *checkpoint* mechanism. Checkmate [5] tries to solve a tensor rematerialization problem through linear programming and minimizes the extra rematerialization costs.

C. Hybrid Tensor Re-Generation

We summarize the most related work in Table I. Both swap and recomputation have been used in many studies, and some studies make effort to use them in a combined manner. However, it is an NP-complete problem even if only considering the optimality of the swap [6]. Although optimization with both swap and recomputation will bring more opportunities, it will also increase the difficulty of finding the optimal solution. Current hybrid solutions are mainly based on some heuristics. For example, SuperNeurons [18] provides *speed-centric* and *memory-centric* strategies to trade off the memory usage and recomputation costs. Capuchin [8] chooses an *eviction candidate set* according to whether the access of a tensor occurs in the peak memory usage period. Afterward, it will try to swap out tensors in the *eviction candidate set* to reduce the peak memory. If the memory reduction still fails to sustain training, recomputation will be used to further evict some tensors.

These ideas combine the advantages of swap and recomputation, that is, the asynchronized swap is preferentially used, and recomputation is used when the use of swap will delay the calculation. However, to simplify the problem, neither SuperNeurons nor Capuchin explored the complete solution space. SuperNeurons chooses to *offload* only the tensors from convolution layers. For Capuchin, the selection of *eviction candidate set* is empirical. Capuchin dynamically adjusts the trigger point of swap-in operations, which tries to seamlessly overlap the swapping time. Besides, prior work did not consider the possibility of fetching a tensor multiple times to take full use of the tensor residing in the host memory. We want to explore a more flexible approach to using swaps, which will be introduced in the following section.

D. Motivation

For nonlinear deep neural networks, the multi-branch architecture [20], [21] has been widely used in current designs. For example, U-Net [22], [23] is a popular architecture in pixel-wise image segmentation tasks, which uses skip connection to combine low-level feature maps with higher-level ones. As shown in Fig. 2, the tensor generated by the MaxPooling2D is not only used by the successor B4-Conv2D, but also used by the downstream Concatenate and two backward nodes. Suppose we use a swap-in operation to regenerate the tensor t before

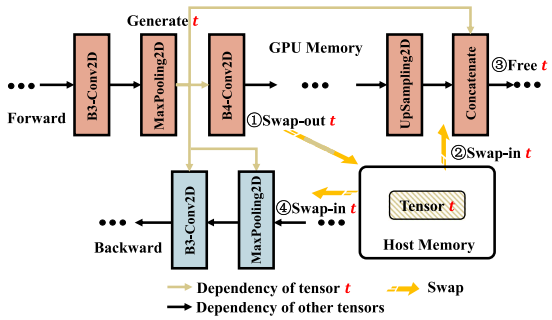


Fig. 2. A part of the architecture of U-Net. Tensor t is optimized for the host checkpoint.

Concatenate. If the tensor can be further freed after Concatenate and swapped into GPU again during the backward access, two recomputations may be eliminated by swapping without the additional computational overhead. We want to promote the previous work by making full use of this characteristic to swap on demand.

Checkpoints in Host Memory: Inspired by the branch architecture mentioned above, we hope to fully utilize the swapped tensor by swapping it back to GPU memory even *multiple times* when needed. It achieves similar functionality to the checkpoint mechanism, and we call it a *host checkpoint*. As illustrated in Fig. 1, we hope to access swapped tensors resided in host memory on demand through the PCIe connection.

In previous work of swapping and hybrid approaches such as SwapAdvisor [16] and Capuchin [8], the swap operation is considered as a *pair* of swapping out and swapping in. To determine *pairs* to swap, Capuchin ranks the pair list by *free time*, which hopes to greedily achieve a longer resident time in host memory, in turn reducing the peak memory of the GPU. When a tensor can be swapped and dropped multiple times, these greedy policies fail to find which tensors can be swapped and when to a swap to avoid the congestion of the limited bandwidth of PCIe. Consequently, a new optimization method is required to consider: (1) which tensors can be swapped, (2) for a specific tensor, how many times the swap-in can be used, and (3) when the swap-in needs to be triggered.

Considering Swap and Recomputation in a Combined Manner: Prior work of the hybrid approach considers the swap and the recomputation individually. For example, SuperNeurons [18] swaps the tensors generated by convolutional layers. SuperNeurons chooses to recompute low-cost operations such as pooling and batch-normalization. Capuchin [8] chooses the swapped tensors from an eviction candidate set, then the recomputation is considered only when the swapping cannot bring enough spare memory. Although these heuristics simplify the optimization problem but may compromise the optimality of the solution. In this paper, we prefer to *formalize a GPU memory optimization problem with the use of swap and recomputation uniformly, and try to find the optimal solution with the lowest computational cost in the scenario of limited GPU memory*. In the following, we will introduce how we formalize our problems and design constraints to achieve our target.

III. Swap-Dominated Tensor Re-Generation

A. Problem Definition

A DNN training procedure can be represented as a computational graph. A tensor-based computational graph $G = (V, E)$ extracted from a deep neural network is a directed acyclic graph with n nodes $V = \{v_1, \ldots, v_n\}$. Each node v_i represents a feature map (i.e., tensor) of the operation i. Nodes are numbered according to a topological order. Edges represent the computation dependencies between tensors. The memory cost to store tensor v_i is denoted as M_i and the computation costs to generate tensor v_i is denoted as C_i. Considering the memory limitation of GPU devices, we need to ensure that the peak memory usage is under a memory budge M_{budget} during a training iteration.

The execution of the computational graph can be divided into n stages according to n operations. An $n \times n$ matrix R filled with binary variables $R_{t,i} \in \{0, 1\}$ are used to represent whether node i will be executed to generate v_i in stage t. When GPU memory is insufficient or using a too-large batch size for training, temporarily unused tensors are dropped to make room for the computation. These freed tensors need to be re-generated through swap or recomputation when downstream operations use them as dependencies. Our target is to find the optimal tensor re-generation solution which achieves minimal computation costs:

$$\arg\min_{R} = \sum_{t=1}^{n} \sum_{i=1}^{t} C_i \cdot R_{t,i} \tag{1a}$$

subject to

$$\sum_{t=1}^{n} R_{t,t} \geqslant n \tag{1b}$$

$$R_{t,i} \in \{0, 1\}, \qquad \forall t \forall i \tag{1c}$$

It is an integer linear programming problem to minimize the total cost. In the subsequent, we introduce how to limit R by constraints. Afterward, the problem can be solved by an out-of-the-box optimizer such as Gurobi [24] and CPLEX [25].

B. Swap Representation

The swapping technique is based on the memory copy between GPU memory and host memory through links. For example, the PCIe 3.0×16 connection reaches up to 12 GB/s bandwidth in practice [8]. For connections such as the NVLink, bandwidths of up to hundreds of gigabytes per second can be achieved [26], which means that memory copies will be more efficient and provide more opportunities for the swap technique. We denote the connection bandwidth between GPU and CPU memory as B. Users need to specify the bandwidth B according to their hardware.

For a specific tensor, we need to consider when to trigger the swap-in and swap-out operations. The swap of a tensor must be finished before the tensor is accessed as a dependency. Otherwise, the computation of operations will be suspended. We prefer to *seamlessly overlap the swap operations with computations*. To estimate the time cost of the swap, two functions are needed:

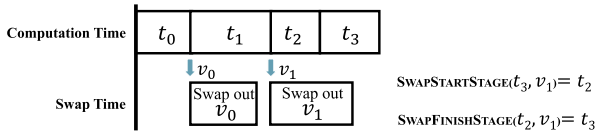


Fig. 3. An example of SwapStartStage and SwapFinishStage.

(1) SwapStartStage(t, i), which returns the stage needed to start swapping tensor v_i in order to finish it in stage t, and (2) SwapFinishStage(t, i) which returns the completion stage of swapping tensor v_i started in stage t. It is worth noting that we always start a swap at the beginning of a stage. Both two functions can be estimated through the size of tensor v_i and PCIe bandwidth.

For a specific tensor i, these two functions may be not calculable for some stages. As an example shown in Fig. 3, tensor v_0 and v_1 need to be swapped out. The swap-out of v_0 can finish in stage t_1 indicates SwapFinishStage$(t_1, v_0) = t_1$ and SwapStartStage$(t_1, v_0) = t_1$. In contrast, the swap-out of v_1 finishes in stage t_3, which indicates SwapFinishStage$(t_2, v_1) = t_3$ and SwapStartStage$(t_3, v_1) = t_2$. In this example, no tensor will complete the swap in stage t_2. To cover such a case, we use SS and SF to simplify the representation:

$$SS(t,i) = \begin{cases} \text{SwapStartStage}(t,i) & \text{function is calculable} \\ \ominus & \text{else} \end{cases} \tag{2}$$

$$SF(t,i) = \begin{cases} \text{SwapFinishStage}(t,i) & \text{function is calculable} \\ \ominus & \text{else} \end{cases} \tag{3}$$

In the following approach, we prepare to use some constraints to guide the solving of the problem, and the constraint will be skipped when the result of $SS(t, i)$ or $SF(t, i)$ is equal to $\ominus$ in this constraint.

C. Dependency Accessible

Different from the vanilla checkpoint mechanism [12], we follow the work of Jain et al. [5] to determine which tensors are checkpoints for each stage in an accurate way. We use a matrix $S_{t,i} \in \{0, 1\}$ to indicate whether the result of operation i is retained in GPU memory in stage t. When computing node j in stage t, dependencies need to reside in GPU memory or be calculated:

$$R_{t,j} \leqslant R_{t,i} + S_{t,i}, \qquad \forall (i,j) \in E \tag{4}$$

We show an example of the matrix S in Fig. 4 and investigate the factors that affect the existence of tensors. At the beginning of the training iteration, there are no tensors existed in GPU memory. Tensor i is generated after the calculation of $R_{i,i}$. We need to determine whether to keep the tensor in GPU memory as a checkpoint or free the tensor. In this example, tensors reside in memory until it is used as a dependency to complete the computations. However, a tensor may be freed when the GPU memory is insufficient and re-generated when needed again. We consider three situations in tensor i can exist in stage $t + 1$: (1) tensor i is already existing in stage t, (2) tensor i is generated by computation in stage t, and (3) tensor i is generated by the *swap-in operation completed in stage* t.

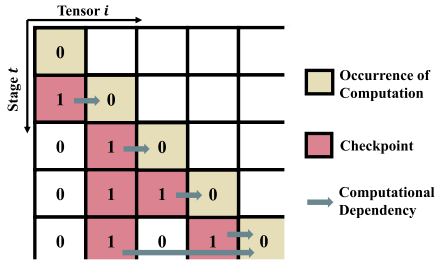


Fig. 4. A sketch for a part of the matrix S. Blocks filled with 1 represent these tensors that will reside in GPU memory. The khaki blocks indicate the computation of the operation, namely, the corresponding position of matrix R will be filled with 1.

To represent the tensor existence, we introduce two matrices P and Q, where $P_{t,i} \in \{0, 1\}$ and $Q_{t,i} \in \{0, 1\}$. P and Q indicate *whether to start* the swap-out and the swap-in operation of tensor i in stage t, respectively. Considering that the time of stages is different because it is determined by operation types. For example, fully-connected layers usually cost more computing time than convolution layers. To fully utilize these stages for swapping, we use a factor Θ_c to represent the percentile for selecting out long stages, where the selected long stages are denoted as L. Every long stage $L' \in L$ will be equally divided into k' *mini-stages*, where $k' = C_{L'}/Average(\{C_i\})$ and $i \in \{V - L\}$. Overall, the shape of matrices P and Q will be $(n + \sum k' - |L|, n)$.

Such a split will make the stage index of P and Q different from matrices R and S. In this paper, we use a mapping function $\gamma(t)$ to express the corresponding stage in matrices R and S when given a stage t in P and Q, and ignore this function when there is no ambiguous within equations. Combining function $SS()$, we have:

$$\sum_{i=1}^{n} S_{1,i} = 0 \tag{5a}$$

$$S_{\gamma(t+1),i} \leqslant S_{\gamma(t),i} + R_{\gamma(t),i} + Q_{SS(t,i),i}, \qquad \forall t, \forall i \tag{5b}$$

where the function $SS(t, i)$ returns the start stage which can complete the swapping in stage t of matrix Q, which makes tensor i accessible in stage $\gamma(t + 1)$ of matrix S. This constraint ensures that a *host checkpoint* can be accessed seamlessly because it has been swapped into GPU or recomputed before being accessed.

D. Correctness of Swapping

According to the swap mechanism, we use constraints to limit the optimization to provide a practicable solution. To achieve the on-demand swapping illustrated in Fig. 1, we have constraints:

$$\sum_{t=1}^{n} P_{t,i} \leqslant 1, \qquad \forall i \tag{6a}$$

$$S_{\gamma(t),i} - P_{SS(t,i),i} \leqslant 0, \qquad \forall t, \forall i \tag{6b}$$

$$P_{SS(t,i),i} + Q_{k,i} \leqslant 1, \qquad \forall i, 1 \leqslant k \leqslant t \tag{6c}$$

$$\sum_{t=1}^{n} P_{t,i} = \begin{cases} 1 & \sum_{t=1}^{n} Q_{t,i} \geqslant 1 \\ 0 & \sum_{t=1}^{n} Q_{t,i} = 0 \end{cases} \tag{6d}$$

in which (6a) limits that there is at most one swap-out operation for each tensor. Compared with GPU memory, host memory usually has a larger space and is cheaper to store tensors. For each iteration, we will always keep the swapped-out tensors in host memory so that they can be accessed again by swap-in operations. The precondition of swapping out a tensor is that the tensor remains in GPU memory, therefore we have (6b). (6c) means that swap-in operations only can be triggered after the swap-out operation is completed before. Equation (6d) indicates that if a tensor i has been swapped out (i.e., $\sum_{t=1}^{n} P_{t,i} = 1$), it must be accessed by swap-in in the following stages, otherwise, the swap-out operation for tensor i will be redundant. The above constraints ensure the correctness of the *host checkpoint*.

According to previous research, a memory copy operation for a single direction will exclusively drain the bandwidth resources [8]. We describe how STR takes advantage of the PCIe bandwidth when swapping. Because both the swap-out and swap-in operations are subject to such a bandwidth constraint, we use X to denote the matrices P and Q. Variables are restricted from two dimensions: *inter-tensor constraint* and *intra-tensor constraint*. The *inter-tensor constraint* means that swap operations of different tensors cannot be performed in the same stage, which follows:

$$X_{t,i} \leqslant \sum_{m=t}^{SF(t,i)} \sum_{i=1}^{n} X_{m,i} \leqslant (1 - X_{t,i})(SF(t,i) - t) + 1, \forall t, \forall i \tag{7}$$

The *intra-tensor constraint* means that for any tensor, the solution cannot include overlapping swap stages, which follows:

$$X_{t,i} \leqslant \sum_{m=t}^{SF(t,i)} X_{m,i} \leqslant (1 - X_{t,i})(SF(t,i) - t - 1) + 1, \forall t, \forall i \tag{8}$$

Theorem 1: If (6), (7) and (8) hold for all t, i of matrices P and Q, then we can make sure that each swap operation does not interfere with the others.

Proof: Consider the matrix Q. For (7), when $Q_{t,i} = 1$, the upper bound on the right side equals to 1, which also limits $\sum_{m=t}^{SF(t,i)} \sum_{i=1}^{n} Q_{m,i} = 1$. We illustrate such a case in Fig. 5(a), where the sum of grey-colored blocks (shaded area) is limited to 1. This means that the swap-in operation occupies the bandwidth resources of the shaded stages. When $Q_{t,i} = 0$, the upper bound becomes the maximum times of using swap-in operations during these stages, namely $SF(t,i) - t + 1$. The (8) corresponds to Fig. 5(b), which has a similar form. The above demonstration is also suitable for matrix P. Combining with (6) that restricts the order of swap-out/in operations, at most one swap operation can occupy the PCIe bandwidth in a stage. ■

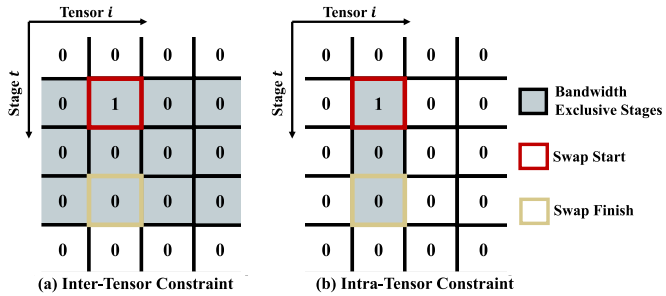


Fig. 5. Sketches of the *inter-tensor constraint* and the *intra-tensor constraint*, they are both part of the matrix Q.

E. Memory Limitation

The peak memory is limited by the size of the GPU memory. An $n \times n$ matrix $U_{t,k} \in \mathbb{R}_+$ with continuous variables is introduced to represent the memory used after computing node k in stage t. For each stage, in addition to the fixed memory size for gradients, parameters and the input mini-batch data, three factors dynamically affect the memory usage: (1) checkpoint tensors determined by S and recomputed tensors determined by R, (2) the memory occupied by unfinished swap-in operation, and (3) the memory reduction caused by freeing tensors. It is worth noting that before the swap-in operation begins, a memory buffer on the GPU device with the same size as the swapped tensor has been allocated. Therefore, we add this size to $U_{t,k}$ when the tensor begins to swap in.

To represent the occupied memory of the swap-in operation in stage t, we introduce a function $\mathsf{Occupied}(t)$ to find a set of (t', i'), in which starting swap-in tensor i' in stage t' (i.e., $Q_{t',i'} = 1$) will occupy the bandwidth of stage t. For each stage t, we have:

$$U_{\gamma(t),0} = M_{input} + 2M_{param} + \sum_{i=1}^{n} M_i S_{\gamma(t),i} + \sum_{(t',i') \in \mathsf{Occupied}(t)} Q_{t',i'} M_{i'} \tag{9}$$

where the M_{input} represents the size of input mini-batch data, and $2M_{param}$ represents the sum of the size of parameters and their gradients. The memory usage of checkpoints is also considered. Afterward, we evaluate the memory usage for all tensors of stage t recursively:

$$U_{t,k+1} = U_{t,k} - MemFreed_t(v_k) + R_{t,k+1} M_{k+1} \tag{10}$$

where $MemFreed_t(v_k)$ is the amount of memory reduced in stage t by freeing tensor v_i and its parents. We use $\mathrm{DEPS}[k] = \{i : (v_i, v_k) \in E\}$ and $\mathrm{USERS}[i] = \{j : (v_i, v_j) \in E\}$ to represent the parents and children of a node separately. After the tensor k is computed, if the tensor i is not a checkpoint in the next stage and there is no successor to rely on it, then it can be deallocated. We introduce a matrix $\mathrm{FREE}_{t,i,k}$ for $(v_i, v_k) \in E$ to denote whether v_i will be deallocated in stage t after the computation of node v_k. Therefore, we have

$$MemFreed_t(v_k) = \sum_{i \in \mathrm{DEPS}[k] \cup \{k\}} M_i * \mathrm{FREE}_{t,i,k} \tag{11}$$

$$\text{FREE}_{t,i,k} = R_{t,k} * (1 - S_{t+1,i}) \prod_{j \in \text{USERS}[i], j > k} (1 - R_{t,j}) \tag{12}$$

(10), (11) and (12) refer to (3)–(5) in Checkmate's paper [5], where (12) is also converted to a linear form. We put them here to keep our method intact and comprehensible.

F. Constraint Linear Reformulation

(6d) is an indicator constraint, we hope to convert it into a linear form, which can be optimized by most optimizers. Adding an upper bound to reformulate the indicator constraint is a commonly used approach, such as the *big-M* formalization [27]. For solving efficiency, we introduce an upper bound η to restrict the maximum number of times the swap-in can be used.

Theorem 2: After limiting the maximum number of swap-in times, (6d) can be converted to the following form.

$$\sum_{t=1}^{n} P_{t,i} \leqslant \sum_{t=1}^{n} Q_{t,i} \leqslant \eta \sum_{t=1}^{n} P_{t,i} \tag{13}$$

Proof: For the case when $\sum_{t=1}^{n} P_{t,i} = 1$, matrix Q is limited with $1 \leqslant \sum_{t=1}^{n} Q_{t,i} \leqslant \eta$, which indicates that $\sum_{t=1}^{n} Q_{t,i} \in [1, \eta]$. For the case when $\sum_{t=1}^{n} P_{t,i} = 0$, $\sum_{t=1}^{n} Q_{t,i}$ is tightly restricted to 0. ■

We reformulate our problem definition in Section III-A with full constraints:

$$\arg\min_{R,S,U,P,Q,\text{FREE}} = \sum_{t=1}^{n} \sum_{i=1}^{t} C_i \cdot R_{t,i}$$

$$\text{subject to} \tag{14a}$$

$$(1b), (1c), (4), (5), (6)$$

$$(7), (8), (9), (10), (11)$$

$$(12), (13), U_{t,k} \leqslant M_{budget}$$

IV. Relaxation Strategy

To ensure the correctness of swapping and computation, we introduce several binary variables and constraints. They help us find better solutions but also greatly increase the solving time cost because of the NP-completeness [28]. The linear programming (LP) problem has been studied for years and is known that it can be solved in polynomial time [29], [30]. Relaxing integrality constraints is a common and effective way to convert integer linear programming to an LP problem.

In our problem, we change the type of R, S, P and Q from binary variables $\{0, 1\}$ to continuous variables belonging to [0,1]. Then the solver (e.g., Gurobi) optimizes the LP problem in polynomial time and returns a solution with continuous values denoted as R', S', P' and Q'. Because of the relaxation, constraints may cannot tightly restrict the solution. For example, a variable in S' may be equal to 0.66, but we need to determine whether it is a checkpoint. In the following, we introduce a *recursive source tracing* algorithm, which makes sure the solution is valid.

Algorithm 1: Recursive Source Tracing.

Input: R', S', P', Q' and U filled with continuous values;
Output: R^I, S^I, P^I and Q^I filled with rounded binary values;

```
1  Initialize tracepath as a list;
2  Initialize R^I, S^I, P^I and Q^I as ⌊R'⌋,⌊S'⌋,⌊P'⌋ and ⌊Q'⌋;
3  Function SourceTracing(t, i):
4      if t ⩽ i then
5          return False;
6      if S^I[t − 1, i] == 1 then
7          SourceTracing(t − 1,i) ;      // Exist before
8      if R^I[t − 1, 1] == 1 then
9          return True ;      // Generated by (re)comput.
10     if Q^I[γ^{-1}(SS(t − 1, i)), i] == 1 then
11         return True ;              // Generated by swap
12     if Cacheable(t − 1,i,U) then
13         Append (t − 1, i) to tracepath;
14         return SourceTracing(t − 1, i);
15     else
16         return False;
17 Function ScheduleSwapOut(P^I, Q^I, S^I):
18     for each swapped tensor i in Q^I do
19         if tensor i has been swapped out in P^I then
20             continue;
21         t = stage where tensor i is first-time generated in S^I;
22         while bandwidth during [t, SF(t, i)] is not empty do
23             t = t + 1;
24         end
25         Set P^I[t, i] to 1;
26     end
27     return P^I;
28 for stage v = 0 to n do
29     for all (u, v) ∈ E do
30         if SourceTracing(v,u) then
31             Fill S^I with 1 indexed by tuples in tracepath if
                tracepath is not empty;
32         else
33             Failed to solve the dependency, continue for the
                next edge;
34         Empty the tracepath;
35     end
36 end
37 P^I = ScheduleSwapOut(P^I, Q^I, S^I);
38 return R^I, S^I, P^I and Q^I
```

We detail the strategy in Algorithm 1, the key idea is to figure out the rationale for the existence of tensors. Denoting that R', S', P', Q' and U are matrices solved with the integrality constraint relaxation. We floor each value of the solution matrices and denote them as R^I, S^I, P^I and Q^I (line 2). A function SourceTracing(t, i) is defined to check the validity of the existence of tensor i in stage t for matrix S^I. There are 3 cases where the existence of a tensor at stage t is reasonable, that is, the tensor already exists before stage t (lines 6-7), is computed (lines 8-9) or is swapped at stage t (lines 10-11). Considering the mini-stage split, we use function $\gamma^{-1}(t)$ to represent the mapping from stages of matrices R and S to a set of stages in Q. If we cannot find plausibility for the existence of a tensor, we will try to mark it as a checkpoint first (lines 12-14). The function Cacheable(t, i, U) indicates whether it is possible to cache tensor i in stage t under current memory usage U, which tries to repair missing dependencies. There are two cases that SourceTracing(t, i) fails to repair the dependency, namely the traverse has reached stage i and the memory is insufficient to

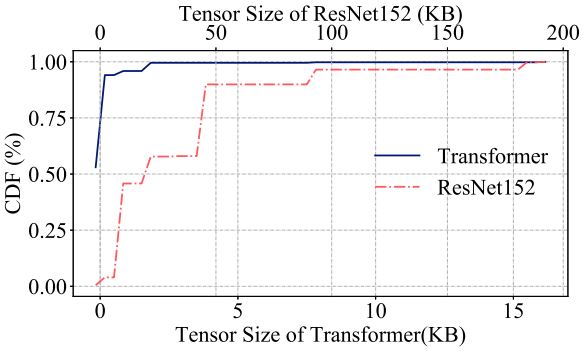


Fig. 6. Distributions of tensor sizes for different models.

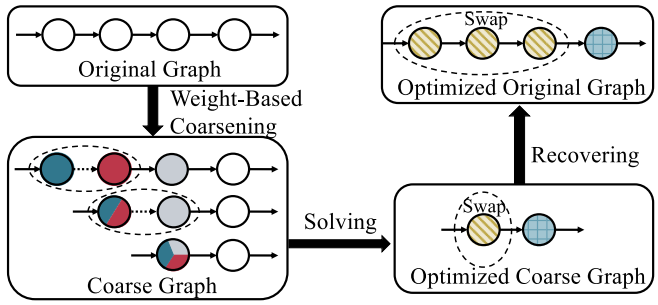


Fig. 7. Steps of reducing the scale of computational graph.

cache tensor i at stage t. The source tracing procedure makes sure that dependencies can be accessed in GPU memory. Another function ScheduleSwapOut(P^I, Q^I, S^I) handles cases that swap-in operations miss their swap-out operations (lines 17–27).

For each computation of $(u, v) \in E$, the tensor u needs to be guaranteed to exist in GPU memory when used as a dependency by tensor v (lines 28-36). SourceTracing can be recursively invoked to check and correct the matrix S^I. If tensor u cannot exist reasonably (i.e., SourceTracing returns *False*), we choose not to optimize tensor u to avoid the dependency missing, which is a compromise for the functionality. Finally, we repair matrix P^I with the function ScheduleSwapOut.

Theorem 3: Algorithm 1 ensures that the solution with constraint relaxation will not degrade the training performance compared with the original training when the memory is insufficient.

Proof: Consider the worst case that all values in matrices R', P' and Q' are less than 1 and floored to 0. The missing swap and recomputation operations lose the ability to break the memory wall, which degenerates to normal training. For other cases, the swap and recomputation operations will spare a specific memory size with guaranteed node dependencies. ■

V. Graph Coarsening Approximation

With the increase of the model scale, the constraint relaxation still needs to spend too much time to finish optimization. We prefer to introduce another approximation method based on Algorithm 1. Benefiting from that the relaxation can guarantee that the training performance will not be degraded after the optimization, we want to coarsen the computational graph to reduce the scale and recover it after the optimization with the constraint relaxation.

Fig. 7 shows the approximation procedure. The original graph will be simplified by a *weight-based coarsening*. This step reduces the number of nodes and keeps the relations between adjacent tensors. Then the *solving* will generate the tensor re-generation solution and the *recovering* maps the optimized solution to the original graph.

A. Weight-Based Coarsening and Solving

We propose to coarse the computational graph based on observations. We illustrate the tensor sizes of Transformer and ResNet152 in Fig. 6 (settings are detailed in Section VII) and notice that there are a lot of small tensors. Some small tensors are adjacent in the computational graph. If we can treat a series of adjacent small tensors as a single chunk, the graph scale will be largely reduced. Therefore, weighted coarsening uses a heuristic that adjacent small tensors can share the same optimization method. From this intuition, we try to select tensors based on the "edge weight" to coarse the computational graph.

Tensors with small sizes usually require less computation and swap costs. Fusing nodes based on the tensor size brings some benefits. For example, instead of triggering only one swap operation at a stage, we can trigger fine-grained swaps of small tensors more intensively to improve bandwidth utilization. To find a set of tensors that can be coarsened, we define a weight for each edge according to the tensor memory consumption.

$$W_{i,j} = M_i + M_j, \forall (i, j) \in E \tag{15}$$

The coarsening procedure will repeatedly select an edge with minimal weight and fuse two connected tensors of the edge. The dependencies of fused tensors will also be re-connected to the new tensor. The computation and memory costs of the new tensor are also set to the sum of two fused tensors. This process will repeat until the number of nodes shrinks to a specific graph size B_{target}. Empirically, we have:

$$B_{\text{target}} = \lfloor n^{0.7} \rfloor \tag{16}$$

Graph coarsening tends to eliminate smaller tensors in the computational graph. The coarse graph will have fewer nodes, which is much easier to optimize. Benefiting from our strategy with the constraint relaxation introduced in Section IV, the coarse graph can be optimized efficiently. After the optimization, a swap and recomputation in generated matrices may correspond to multiple nodes in the original graph. We will recover the solution from the coarse graph to get the practicable operations for the original graph.

B. Graph Recovering

To recover the graph from the optimized solution, swaps and recomputations need to be considered separately. Assuming that tensors $\{v_i, \ldots, v_j\}$ in the original graph are fused to $\mathbb{v}$, which will be swapped during specific stages $[\mathbb{t}, \mathbb{t}']$ in the optimized coarse graph. We denote the corresponding stages in the original graph as $[t_m, t_n]$. Constraints in Section III-D ensure that the

swap of tensor $\mathbb{v}$ can be finished during $[\mathbb{t}, \mathbb{t}']$. Theoretically, $\{v_i, \ldots, v_j\}$ also can be swapped during stages $[t_m, t_n]$. However, prior studies have verified that intensive fragmented swaps cannot fully utilize the bandwidth [16]. For a set of small tensors, triggering swap operations by stages will lead to low bandwidth usage.

For the swap operation, we sort $\{v_i, \ldots, v_j\}$ by the access sequence and trigger swaps in a *chained rule:* a series of swap operations for sorted tensors will be chained through the controlinput function and triggered one by one. This implementation breaks the assumption that only triggers one swap operation within a stage, but it ensures the series of swap operations can be finished during $[t_m, t_n]$. For the recomputation, if $\mathbb{v}$ will be recomputed at stage $\mathbb{t}$ in the optimized coarse graph, the corresponding tensors $\{v_i, \ldots, v_j\}$ will be recomputed as needed during stage $[t_m, t_n]$ for the original graph.

Because multiple nodes in the original graph may be fused to one node based on heuristics, we cannot get a performance bound like methods using constraint relaxation. Our experiments show that such a coarsening-optimizing-recovering procedure still brings significant performance improvements. Besides, compared with the fragmented swapping, packing tensors can further improve the efficiency of copying memory in prior studies [31], and we leave this improvement as future work.

VI. Implementation and Workflow

Profiling: According to our problem definition, the computation time and memory usage of each layer are required to solve the optimization problem. The *tensorboard-plugin-profile* tool [32] is employed to collect runtime statistics. After the first 10 iterations of the model training warm-up, we collect the execution time for each layer in milliseconds, corresponding to C_i in (1a). Another choice is estimating the execution time by different types of operations in the neural network. However, the performance of an operation is diverse on different GPU devices. Thus, profiling is recommended and helps STR generate better solutions. We also collect the memory cost M_i for each layer based on the size of parameters, gradients and feature maps. These statistics will be queried when optimizing the corresponding neural networks.

Optimization: For the optimization procedure, we implement STR's optimizer by the Python API of Gurobi [24], a widely-used mathematical optimizer. We initialize R, S, P and Q as a lower triangular matrix according to our definition. By giving a memory budget and constraints in (14), Gurobi will find the optimal solution according to our variables and constraints. After the optimization, we store optimized solutions (i.e., matrices) to be used at runtime.

Graph Editing: To use the optimized strategies, we implement the runtime of STR in TensorFlow [33] based on editing the computational graph [9], [17]. In TensorFlow Keras APIs, *Optimizer* handles the graph construction. We manipulate the process of the generation of the gradient nodes to achieve the recomputation. The recomputation nodes will be inserted along with normal backward operations according to matrix R. Then we further add swap out/in operations and control operations to achieve swapping. A swap-out operation will create a new tensor in CPU memory via tf.identity based on the tensor being swapped out, and a swap-in operation does the opposite. We utilize the *control input* functionality to take advantage of matrices P and Q that indicate the swap trigger points. The dependencies of operations affected by swapping will also be updated to maintain the computational logic. Finally, the modified graph will be fed to the graph scheduler for execution.

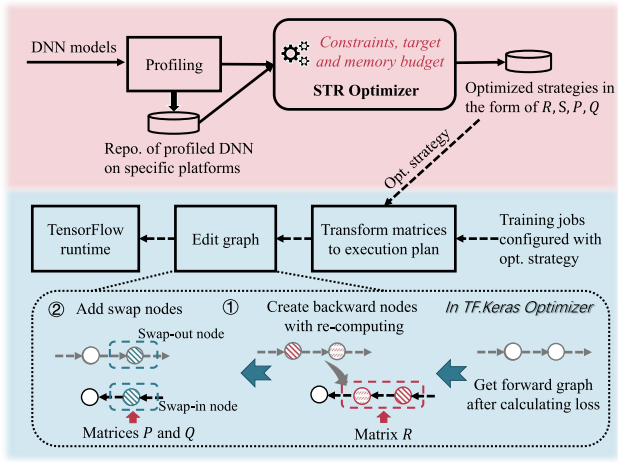


Fig. 8. The workflow of STR. The solid lines and dotted lines represent the optimization steps and runtime steps accordingly.

We illustrate the workflow in Fig. 8. For a specific model, we only profile for the first time. For training jobs without profiling, we assume that the computational cost linearly increases with the batch size, and employ a simple linear regression model to estimate the computational cost C_i. The *STR Optimizer* will query the profiling statistics and produce the optimization solution. After the optimization, the solution can be loaded for the TensorFlow runtime. When a training job is configured with a solution, we automatically transform these matrices to recomputation and swapping behaviors, and edit the computational graph accordingly. These steps are transparent for users and easy to be used.

VII. Evaluation

Experimental Setup: We implement our prototype of STR on TensorFlow-1.15.2 [33]. Gurobi-9.1.2 [24] is used as STR's optimizer. We use Python 3.6 to implement the optimizer on Gurobi and the runtime on TensorFlow. For the system environment, we run our experiments on a server equipped with two 14-core Intel Xeon Gold 5117 CPUs and 256 GB DDR4 host memory. An NVIDIA TITAN RTX GPU card (CUDA version is 10.2) with 24 GB memory is equipped. Based on our testbed, we set the bandwidth B to 10 GB/s in our evaluations.

Training DNNs: We choose 5 widely used deep neural networks implemented on TensorFlow and listed in Table II. They achieve high accuracies on common tasks such as image classification, segmentation and text classification. It is representative to choose them because many follow-up studies have used similar architectures.

Baselines: We choose 5 baselines to evaluate STR. The first is a heuristic approach presented by Chen et al. [12] (denoted

TABLE II
DETAILS OF DNN MODELS

Models	Tasks	Datasets
VGG16 [34]	Image classification	CIFAR-10 [35]
U-Net [22]	Face detector	Face landmark [36]
MobileNet [37]	Image classification	CIFAR-10 [35]
ResNet101 [20]	Image classification	CIFAR-10 [35]
ResNet152 [20]	Image classification	CIFAR-10 [35]
Transformer [38]	Sentiment classification	IDMB [39]

as *Chen-heurist*). To compare with our work, we use an implementation of this work from Cybertron AI [9]. The second is Checkmate [5]. Checkmate formalizes the recomputation problem into a mixed integer linear programming problem, which also produces matrix R for recomputation. The third is Capuchin [8], which is the state-of-the-art hybrid approach that uses both swap and recomputation techniques with inspired observations. We re-implement Capuchin's strategy (which code is unavailable) to generate matrices R, P and Q. The optimized tensor of Capuchin is selected according to the peak memory of training. For a specific DNN with different batch sizes, the peak memory is also various. In our experiment, we use a threshold 0.7 to select the *eviction candidate set*. That means if a tensor is accessed when the memory is greater than 70% of the peak memory, we move it to the *eviction candidate set*. The fourth is DYNPROG [6], which uses a dynamic programming algorithm to find the optimal offloading strategy. We also convert the strategy generated by DYNPROG to matrices that can be recognized by STR's TensorFlow runtime components. In addition to the above four optimization methods, we also provide the performance of original TensorFlow denoted as "Origin-TF". For 2 factors of STR, we use empirical values in this paper. We set η introduced with (13) to 5, which means at most 5 swap-in operations will be used for a tensor. The factor Θ_c in Section III-C filters out long stages by setting it to 95-th percentile. Two factors can be further tuned by manual or automatic methods [40], [41]. For STR, we denote the solution generated by the constraint relaxation algorithm as STR-app. The graph coarsening approximated solution is denoted as STR-app-gc.

Methodology: To evaluate the end-to-end system performance, we use training throughput as the performance metric to compare the effectiveness of STR with other baselines. Models configured with different batch sizes are trained to simulate the cases with different levels of memory insufficiency. The time cost of optimization is also an important factor of usability. We compare the optimization time cost of STR and its two approximation methods to provide a best practice for choosing different approximations according to model scales.

In this section, we will evaluate the end-to-end runtime performance of STR in Section VII-A and Section VII-B. The time cost of optimization is verified and discussed in Section VII-C. Then, we further discuss why STR offers better solutions and its limitations in Section VII-D.

A. Performance Improvement of STR

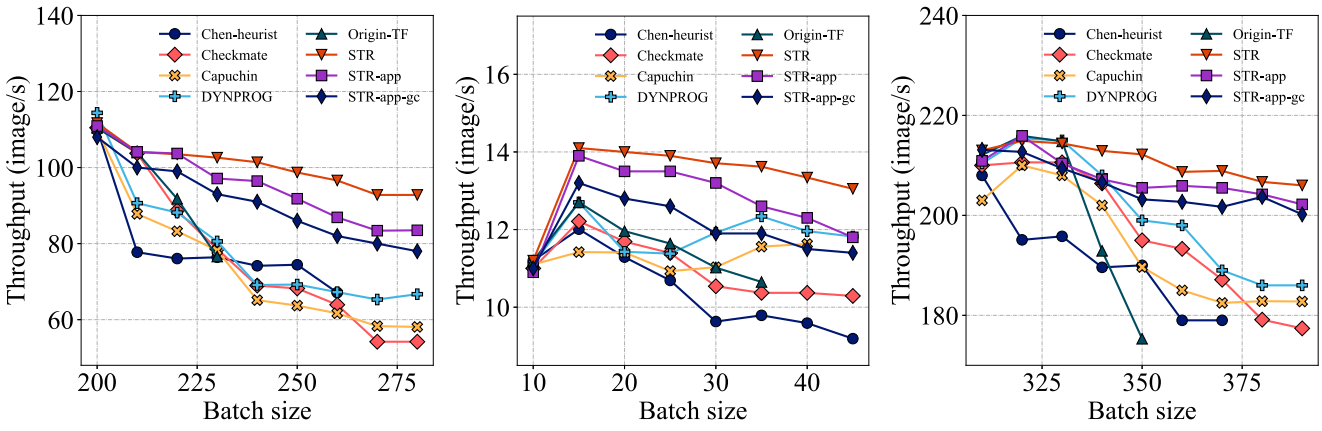
We compare the performance between STR and all baselines on VGG16, U-Net and MobileNet. The graph coarsening approximated solution proposed in this work (denoted as STR-app-gc) is also compared with our previous paper [11] and its performance will be discussed later.

STR achieves higher throughput when the GPU memory is insufficient: For each model, we choose different sizes of the training batch to compare the performance of STR with the other 5 baselines. The training throughput of STR and other baselines are illustrated in Fig. 9. Among three models in Fig. 9, STR improves the throughput by 23.8%, 22.2%, 22.7% and 16.1% on average compared with Chen-heurist, Checkmate, Capuchin and DYNPROG, respectively.

We provide a deeper analysis of the result. In cases with smaller batch sizes, all solutions have similar throughput. Chen-heurist shows lower throughput because it automatically selects tensors to recompute even if there has enough GPU memory, which brings extra computations. As the growth of the batch size, Origin-TF will fall back to a slower implementation or try to free some spaces before crashing when the memory is insufficient. Benefiting from memory optimization strategies, other optimized solutions can support a larger batch size, but throughputs also have degradations because of the limited computing power. For memory insufficient cases, Chen-heurist and Checkmate will generate strategies with more recomputation operations. Despite such a strategy alleviating memory insufficiency, the extra computation overhead reduces the performance. For example, for batch size 390 of MobileNet, STR chooses 6.4% tensors for recomputation and 88% tensors for swapping, while Checkmate chooses 33.6% tensors for recomputation. Capuchin's hybrid solution relies on the selection of *eviction candidate set*, which is hard to achieve the optimum. DYNPROG shows better performance because it can find the exact swapping tensors which achieve the minimum idle time for operations. However, it doesn't consider when to trigger the swap-out and swap-in operations. Of special interest is that the tensor selected by STR covers 100% and 92.7% tensors selected by DYNPROG and Capuchin, separately. The appropriate selection of tensors and trigger points brings performance improvements for STR.

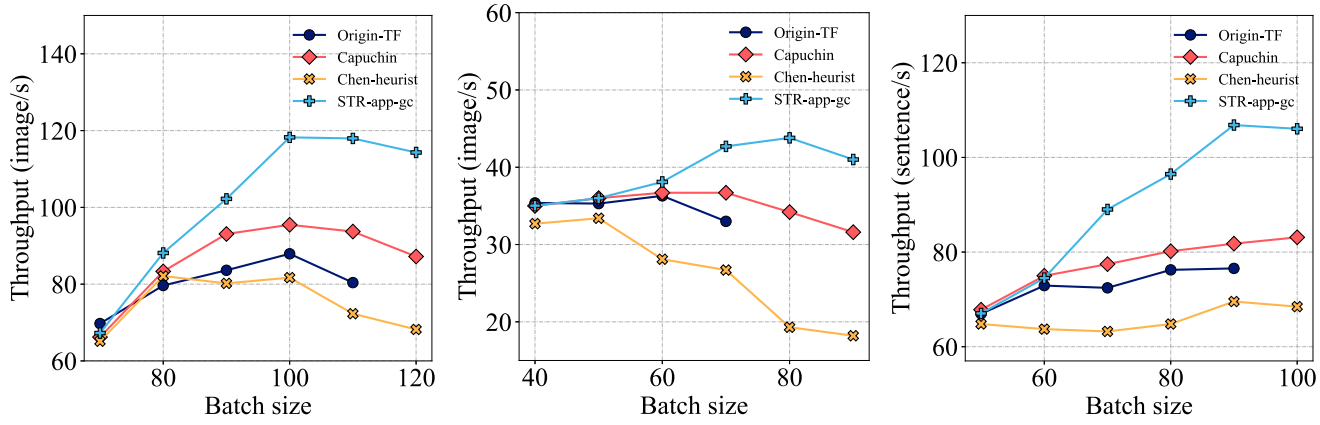
The performance penalty of the approximation solution is limited: The constraint relaxation (denoted as STR-app) can significantly reduce optimizing time cost, but it will bring about a loss of the optimization effect. We also illustrate the loss of the throughput optimized by STR-app in Fig. 9. We find that for VGG16, U-Net and MobileNet, the solutions generated by STR-app have only 5.3%, 4.8% and 2.4% performance losses on average compared with accurate solutions of STR, and achieve better performance than other baselines in most cases. DYNPROG slightly outperforms STR-app on U-Net when batch size is large because of the rounding of Algorithm 1. Compared with our preliminary work [11], STR-app on VGG16 and U-Net performs better because we remove the greed repairing with recomputations.

Constraint relaxation is better for optimizing moderate-scale models than graph coarsening: The approximation with graph coarsening is also depicted as STR-app-gc in Fig. 9. The most obvious phenomenon is that graph coarsening will bring performance degradation when the graph scale is not large. For example, STR-app-gc losts 9.6% and 9.8% performance on



(a) VGG16 (224×224) (b) U-Net (320×640) (c) MobileNet (224×224)

Fig. 9. Comparing the throughput under different batch sizes. Missing points represent OOM that occurred during training. The input resolution is labeled after the model name.



(a) ResNet101 (224×224) (b) ResNet152 (224×224) (c) Transformer (max len. of words=128)

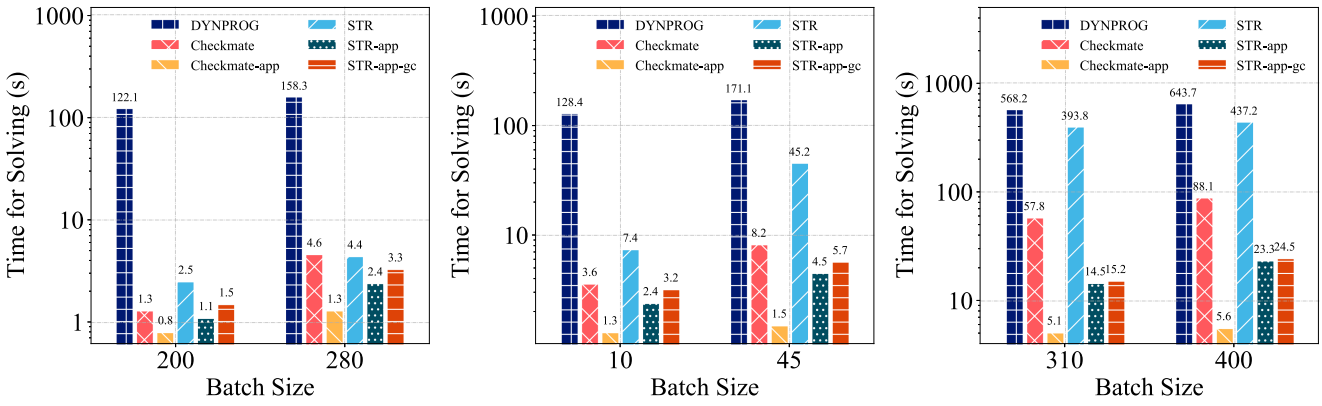
Fig. 10. Performance with graph coarsening approximation. Missing points represent training crashes caused by memory limitations.

average compared with our accurate solution for VGG16 and U-Net respectively. For MobileNet which has a larger graph scale, STR-app-gc achieves only 2.3% performance loss. It means that graph coarsening approximation performs better for cases where models contain more layers. Next, we investigate graph coarsening approximation on large models separately and give a best practice for choosing different approximations.

B. Optimizing Large Models

Because the series of ResNet and Transformer models have larger graph scales, DYNPROG, Checkmate and Checkmate-app fall short of completing the optimization within 8 hours in our experiments. STR with graph coarsening (STR-app-gc) can still support this situation by reducing the graph sizes. We compare STR-app-gc with a part of baselines that can handle large models at an acceptable time cost, namely Capuchin and Chen-heurist, as well as the Origin-TF. We choose ResNet101, ResNet152 and Transformer to show the effectiveness of graph coarsening approximation for large models. Specifically, the Transformer model in our experiment is configured with 6 encoders and 6 decoders which refers to the setting of [42]. We use 2 heads and 8 model dimensions for the Transformer according to our hardware.

STR with graph coarsening significantly accelerates large models: As can be seen in Fig. 10, the training throughput after approximation can be up to 25.8%, 28.1% and 30.6% higher than Capuchin for ResNet101, ResNet152 and Transformer models, respectively. The graph coarsening procedure effectively reduces the graph scale while keeping the tensor dependencies. The numerical optimizer can easily solve the linear programming problem with constraint relaxation. Compared with the training without optimizations, Capuchin also achieves higher throughput when the GPU memory is insufficient. However, the empirically configured parameters cannot be well adapted to various DNN models. For Chen-heurist, it improves the maximum batch size of training, but the recomputation overheads prevent the increase of the training throughput.



(a) Optimization for VGG16 (b) Optimization for U-Net (c) Optimization for MobileNet

Fig. 11. We compare the optimization time cost of three approaches that generate exact solutions. The time cost is numbered on the bar.

C. Time Cost of Optimization

The scale of the neural network and the size of the training batch affect the size of the solution space, which significantly affects the time cost of optimization. We investigate the time cost reduction brought by STR-app and STR-app-gc separately.

The optimization time cost of STR is acceptable: We expose the time costs of configuring the minimum and the maximum batch sizes when optimizing the 3 networks in Fig. 9. We first compare STR with 2 exact solutions, Checkmate and DYNPROG. For methods that employ constraint relaxation, Checkmate-app, STR-app and STR-app-gc are also compared. As illustrated in Fig. 11, for 3 neural networks, DYNPROG spends more time on generating the optimized strategy. This is because its dynamic programming algorithm grows the search space exponentially according to the number of network layers. For VGG16 and U-Net, both STR and Checkmate can finish the optimization in seconds. It needs to be noted that STR will spend more time compared with Checkmate because it uses more variables and constraints to represent swap primitives. Fortunately, the constraint relaxation dramatically reduces STR's solving time to dozens of seconds. Compared with STR, STR-app reduces 50.7%, 78.8% and 95.5% optimization time for VGG16, U-Net and MobileNet, respectively. The time cost of STR-app-gc is slightly larger than STR-app because of the additional graph coarsening and recovering. Considering the NP-hardness of the problem, it is acceptable to spend such an optimization time cost on a few days of training.

STR with graph coarsening optimizes large models efficiently: For large DNN models such as Transformer, all exact solutions fall short of optimizing such a large computational graph. Only Capuchin and Chen-heurist in our baselines can provide optimization solutions with acceptable time costs. In the former experiments, DYNPROG shows comparable performance in many cases, but its high time costs become a barrier to optimizing large models. Fig. 12 illustrates the average time costs of optimization of experiments in Fig. 10. The subfigure shows that the most prominent cost is still the solving time. The coarsening and recovering only take 3.2%, 3.5% and 4.4% of

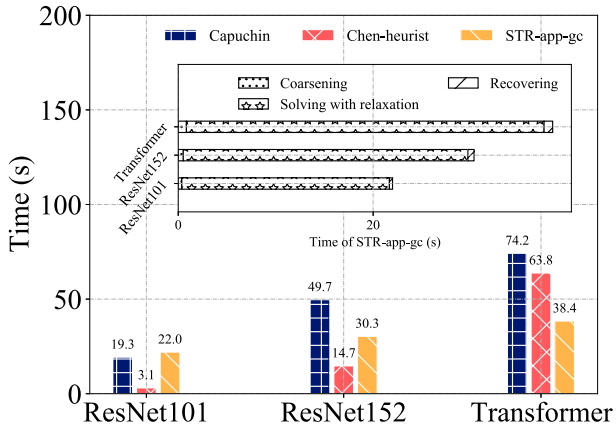


Fig. 12. Time costs for graph coarsening approximation.

the total time costs for optimizing ResNet101, ResNet152 and Transformer, but greatly reduces the difficulty of optimization and significantly improves the training performance. As can be seen, the time costs of Capuchin and Chen-heurist are largely increased with the model scale, since they will search n tensors and their dependencies with $O(n^2)$ complexity. In contrast, the growth of the solving time of STR-app-gc is relatively stable.

Choosing approximations according to model scales: In experimental results in Figs. 9 and 11, the graph coarsening does not perform better compared with STR-app. However, for the experiments of large models in Figs. 10 and 12, the graph coarsening shows the advanced training performance and the stable time cost. Experimentally, we find that *STR with constraints relaxation is suitable for handling models that are not very large to achieve better performance, while the graph coarsening approximation can be specialized for optimizing large models.*

D. Discussion

Various Swap Patterns: We further investigate how STR handles different degrees of memory insufficiency. To simplify the

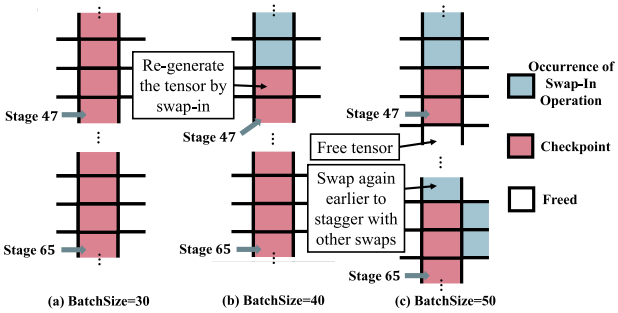


Fig. 13. Behaviors of the tensor 23 in the U-Net computational graph under different batch sizes.

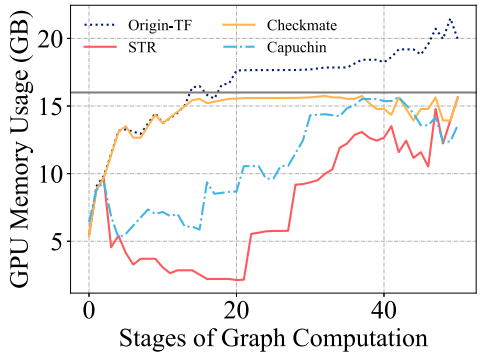


Fig. 14. Analysis of GPU memory usage for a training iteration. The Origin-TF will break the memory budget (set to 16 GB) during training.

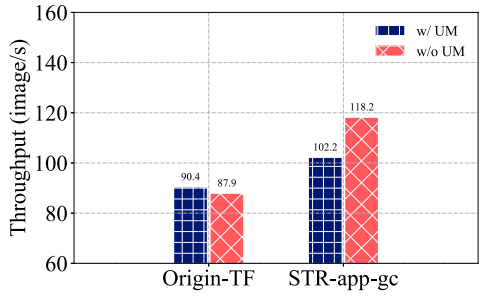


Fig. 15. Performance results of CUDA unified memory on ResNet101 with BatchSize = 100. "w/ UM" and "w/o UM" mean whether to enable unified memory during training, respectively.

case, we disable the mini-stage split (i.e., set $\Theta_c = 100$). The architecture of U-Net [22] contains a sequence of up-convolutions and concatenations, and the output of a concatenation will be used by non-adjacent layers. From the perspective of the computational graph, a tensor generated by the concatenation operation is connected to the latter nodes through branches. The tensor v_{23} is generated by concatenation in stage 23, and is accessed in stages 47 and 65. We take it as an example to show interesting observations.

We optimize U-Net with batch sizes 30, 40 and 50 respectively. Under a specific memory budget, v_{23} shows different behaviors in three solutions, which has been shown in Fig. 13. For BatchSize = 30, v_{23} was always kept as a checkpoint and will not be freed after generation. For BatchSize = 40, it was freed after being used by the successor operation of the forward propagation. According to (4), v_{23} must exist in stage 47. We observe that a swap-in operation is triggered in advance to re-generate it. Then v_{23} was kept in GPU memory until the next access. For BatchSize = 50, v_{23} was also swapped into GPU before stage 47, but after this accessing, it is freed because the memory insufficiency is more serious. Then v_{23} was swapped again, but the swapping is completed in stage 62 because other swap operations that also need the bandwidth resources. STR will automatically consider these situations to generate a plan with the lowest computational cost. We also analyze the swapped tensors for BatchSize = 50, and find that not only all tensors generated by the *convolution* are swapped, but tensors generated by *pooling*, *up-sampling*, *concatenation* and *batch-normalization* also have been swapped. Consequently, STR will find a better solution than heuristic-based approaches that choose only to swap the convolution layers' output.

The Maximum Batch Size Supported: STR cannot improve the maximum batch size compared with Checkmate. We simulate the memory usage of U-Net (BatchSize=40) according to the procedure of graph execution configured with different optimization approaches. We set the memory budget to 16 GB and illustrate the memory usage during a training iteration. As can be seen in Fig. 14, STR achieves the lowest memory usage during the forward pass because it releases more tensors. But at the end phase of the iteration, large and ready-to-use tensors prevent STR to make further optimizations. In our experiments, STR achieves the same maximum batch size as Checkmate for all tested models. Furthermore, in Fig. 9, we observed that when the GPU memory is insufficient, increasing the batch size will lead to a performance loss. We notice that there is still debate on whether a larger batch size has benefits to the quality of the model [43], [44]. For the GPU memory optimization, perhaps it is better to let the user decide whether to maximize the training efficiency or the batch size. We leave it as future work.

Effects of Enabling Unified Memory. CUDA unified memory (UM) [45] enables GPU memory oversubscription for DNN training. It creates a unified logical view of memories across GPU and host, which swaps tensors automatically and reduces the risks of causing GPU out-of-memory. TensorFlow supports turning on unified memory by specifying a configuration of *per_process_gpu_memory_fraction*. To demonstrate the impact of UM, we set this configuration to 2 to double the virtual memory that can be allocated, and compare it with disabling unified memory on ResNet101. As shown in Fig. 15, enabling UM for the original training brings very limited performance improvements but get fewer warning of out-of-memory from TensorFlow. When training with STR-app-gc, enabling UM makes the training throughput downgrade from 118.2 img/s to 102.2 img/s. The reason for performance degradation is that unified memory allocation conflicts with STR's memory planning and invalidates swap operations.

Limitations of STR: We discuss the limitations of STR from two aspects: the competition for bandwidth resources and the dynamicity of models. Exploring the opportunities for hybrid memory optimization of parallel training accelerations is an interesting topic in the future direction. For vanilla data parallel

training, communications between workers are mainly concentrated at the end of each iteration for parameter synchronization. The communications (e.g., all-reduce) will have less impact on swap operations. However, the pattern of communication becomes complicated when using model parallel and pipeline parallel strategies [46]. The scarcity of bandwidth resources has been exacerbated and the difficulty of scheduling memory swapping has increased. Another limitation is that STR works at compile time and falls short of optimizing models with dynamic computational graphs. For example, the computational graph of sparsely-connected models usually is determined at runtime [47], [48]. For deep learning recommendation models (DLRM), the required portions of the embedding table have to be prefetched according to training datasets at runtime [49]. STR needs the static computational graph to plan swap and recomputation operations, and the dynamicity of graphs invalidates the optimized solution. In these situations, using lightweight optimization policies with some heuristics is a more practical solution [50], [51].

VIII. Related Work

Optimization for the Single GPU Training: Training DNN models with a single GPU is a common scenario. We summarize the prior work from two aspects: the *system* and the *model building*. The part of *system* is most relevant to our work. We first overview the studies about *recomputation* or *re-materialization*. For the memory optimization problem, Chen et al. [12] presented a pioneering approach that costs $O(\sqrt{n})$ memory to train an n layer network. Its implementation is known as the gradient checkpointing [9] and brings a wide concern. Gruslys et al. [52] use a dynamic programming approach to select checkpoints. Kusumoto et al. [19] presented a theoretic framework that formalizes the general recomputation problem of minimizing the computational overhead under a fixed memory budget. Checkmate [5] also employed the graph theory to formalize its recomputation problem, which uses the numerical optimizer to find the optimal re-materialization strategy. Mimose [53] is an input-aware checkpointing planner, which eliminated the pre-analyzing process of models by using a lightweight prediction model of GPU memory. XEngine [54] considered a heterogeneous scenario that CPU and GPU both contribute the computing power for training. XEngine solved the rematerialization problem by defining it as a mixed integer quadratic program. Compared to methods based on recomputation or rematerialization, STR further reduces computational overheads through the swap technique.

Another technique *swap* is also intensively studied by researchers. vDNN [14] investigated the using of *offloads* and *prefeches* for tensors that are not immediately required. Meng et al. [7] also presented a simple but effective approach to swap tensors for the Seq2Seq models. Beaumont et al. [6] analyzed the *offloading* problem and proved that it is strongly NP-complete to solve this problem. SwapAdvisor [16] considered the network branches and optimized the problem through the genetic algorithm. TFLMS [17] is an implementation on TensorFlow, which achieves swapping by graph rewriting. In addition, some studies combined these two techniques, such as Capuchin [8] and SuperNeurons [18]. These combined studies inspire us to make a hybrid optimization in STR. The preliminary work [11] of STR uses the swap-out/in operations to achieve the *host checkpoint* functionality. This paper further improves the optimization efficiency, which strengthens the capability for optimizing large models.

For the part of *model building*, there are many optimization techniques have been proposed such as pruning [55] and quantization [56]. These studies are orthogonal to our work.

Optimization for the Distributed DNN Training: Parallel and distributed training of DNN models are also been studied recently. *Data parallelism* [57] is the most common approach to benefit the multiple GPUs. *Model parallelism* also has been used to train large models that cannot be fit into a single GPU, which has been investigated in systems such as [58]. Combining the advantages of these two parallelisms, the *pipeline parallelism* gained widely studied by researchers and produced many systems such as DAPPLE [4] and PipeDream [46]. STR currently considers the single GPU scenario. For distributed training, the competition of swapping between multiple GPUs needs to be reconsidered.

IX. Conclusion and Future Work

For the single GPU memory optimization problem, it is challenging to consider both the swap and the recomputation techniques for training in the memory insufficient scenario. In this paper, we introduce a novel memory optimization strategy called STR, which benefits from techniques including swapping and recomputation. For the memory-limited scenario, the key idea of STR is to re-generate tensors seamlessly through well-designed swap and recomputation operations. We formalize the memory optimization problem into a mixed integer linear programming problem. Some constraints and variables are designed to guide the solving with the help of off-the-shelf numerical optimizers. Afterward, we introduce a constraint relaxation method and an approximation by a graph coarsening procedure. In our experiments, STR improves the throughput by 23.8%, 22.2%, 22.7% and 16.1% on average compared with Chen-heurist, Checkmate, Capuchin and DYNPROG, respectively.

Using swapping to gain an additional performance is particularly appealing but challenging. Our experiments show that swapping is a good choice for improving training performance with the proper overlap of swapping and computation. However, for the scenario of distributed training, complicated communications need considerable bandwidth resources. Considering the trade-off of recomputation and swap, how to co-scheduling swaps and other communications is a problem worthy of further study. In addition, the optimizing and graph editing process of STR can be implemented at the compiler level in the future to facilitate usability.

Acknowledgment

We would like to thank the anonymous reviewers for their insightful comments.

REFERENCES

[1] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of large-scale multi-tenant GPU clusters for DNN training workloads," in *Proc. USENIX Conf. Usenix Annu. Tech. Conf.*, 2019, pp. 947–960.

[2] W. Xu, Y. Zhang, and X. Tang, "Parallelizing DNN training on GPUs: Challenges and opportunities," in *Proc. World Wide Web Conf.*, 2021, pp. 174–178.

[3] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, "Memory-efficient pipeline-parallel DNN training," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 7937–7947.

[4] S. Fan et al., "DAPPLE: A pipelined data parallel approach for training large models," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2021, pp. 431–445.

[5] P. Jain et al., "Checkmate: Breaking the memory wall with optimal tensor rematerialization," in *Proc. 3rd Conf. Mach. Learn. Syst.*, 2020, pp. 497–511.

[6] O. Beaumont, L. Eyraud-Dubois, and A. Shilova, "Optimal GPU-CPU offloading strategies for deep neural network training," in *Proc. Eur. Conf. Parallel Process.*, Springer, 2020, pp. 151–166.

[7] C. Meng, M. Sun, J. Yang, M. Qiu, and Y. Gu, "Training deeper models by GPU memory optimization on tensorflow," in *Proc. ML Syst. Workshop Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1–8.

[8] X. Peng et al., "Capuchin: Tensor-based GPU memory management for deep learning," in *Proc. 25th Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2020, pp. 891–905.

[9] "Gradient checkpointint," 2019. [Online]. Available: http://github.com/openai/gradient-checkpointing

[10] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using megatron-LM," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, pp. 58:1–58:15.

[11] L. Wen, Z. Zong, L. Lin, and L. Lin, "A swap dominated tensor re-generation strategy for training deep learning models," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2022, pp. 996–1006.

[12] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," 2016, arXiv: *1604.06174*.

[13] M. Kirisame et al., "Dynamic tensor rematerialization," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–31.

[14] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, "vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design," in *Proc. IEEE/ACM 49th Annu. Int. Symp. Microarchitecture*, 2016, pp. 18:1–18:13.

[15] S. B. Shriram, A. Garg, and P. Kulkarni, "Dynamic memory management for GPU-based training of deep neural networks," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2019, pp. 200–209.

[16] C. Huang, G. Jin, and J. Li, "SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping," in *Proc. 25th Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2020, pp. 1341–1355.

[17] T. D. Le, H. Imai, Y. Negishi, and K. Kawachiya, "TFLMS: Large model support in tensorflow by graph rewriting," 2018, arXiv: *1807.02037*.

[18] L. Wang et al., "Superneurons: Dynamic GPU memory management for training deep neural networks," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2018, pp. 41–53.

[19] M. Kusumoto, T. Inoue, G. Watanabe, T. Akiba, and M. Koyama, "A graph theoretic framework of recomputation algorithms for memory-efficient backpropagation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 1161–1170.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[21] J. Wang, Z. Wei, T. Zhang, and W. Zeng, "Deeply-fused nets," 2016, arXiv: *1605.07716*.

[22] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," 2015, arXiv: *1505.04597*.

[23] V. Iglovikov and A. Shvets, "Ternausnet: U-net with VGG11 encoder pre-trained on imagenet for image segmentation," 2018, arXiv: *1801.05746*.

[24] "Gurobi," 2023. [Online]. Available: https://www.gurobi.com

[25] "IBM ilog CPLEX optimizer," 2023. [Online]. Available: https://www.ibm.com/analytics/cplex-optimizer

[26] "NVIDIA NVLink connections," 2023. [Online]. Available: https://www.nvidia.com/en-us/data-center/nvlink/

[27] P. Bonami, A. Lodi, A. Tramontani, and S. Wiese, "On mathematical programming with indicator constraints," *Math. Prog.*, vol. 151, no. 1, pp. 191–223, 2015.

[28] H. Shahrabi Farahani and J. Lagergren, "Learning oncogenetic networks by reducing to mixed integer linear programming," *PLoS One*, vol. 8, no. 6 , 2013, Art. no. e65773.

[29] N. Karmarkar, "A new polynomial-time algorithm for linear programming," in *Proc. Conf. Symp. Theory Comput.*, 1984, pp. 302–311.

[30] J. Renegar, "A polynomial-time algorithm, based on newton's method, for linear programming," *Math. Prog.*, vol. 40, no. 1–3, pp. 59–93, 1988.

[31] C. Chu, K. S. Khorassani, Q. Zhou, H. Subramoni, and D. K. Panda, "Dynamic kernel fusion for bulk non-contiguous data transfer on GPU clusters," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2020, pp. 130–141.

[32] "Tensorflow profiler: Profile model performance," 2022. [Online]. Available: https://www.tensorflow.org/tensorboard/tensorboard_profiling_keras

[33] "Tensorflow," 2023. [Online]. Available: https://www.tensorflow.org

[34] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015.

[35] "The CIFAR-10 dataset," 2009. [Online]. Available: https://www.cs.toronto.edu/kriz/cifar.html

[36] "iBUG 300-W face landmark dataset," 2016. [Online]. Available: https://ibug.doc.ic.ac.uk/resources/facial-point-annotations/

[37] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, arXiv: *1704.04861*.

[38] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

[39] "Sentiment analysis of IMDb movie reviews," [Online]. Available: https://ai.stanford.edu/amaas/data/sentiment/

[40] Z. Zong, L. Wen, X. Hu, R. Han, C. Qian, and L. Lin, "MespaConfig: Memory-sparing configuration auto-tuning for co-located in-memory cluster computing jobs," *IEEE Trans. Services Comput.*, vol. 15, no. 5, pp. 2883–2896, Sep./Oct. 2022.

[41] R. Han, S. Wen, C. H. Liu, Y. Yuan, G. Wang, and L. Y. Chen, "EdgeTuner: Fast scheduling algorithm tuning for dynamic edge-cloud workloads and resources," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 880–889.

[42] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–12.

[43] I. Kandel and M. Castelli, "The effect of batch size on the generalizability of the convolutional neural networks on a histopathology dataset," *ICT Express*, vol. 6, no. 4, pp. 312–315, 2020.

[44] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–16.

[45] S. W. D. Chien, I. B. Peng, and S. Markidis, "Performance evaluation of advanced features in CUDA unified memory," in *Proc. IEEE/ACM Workshop Memory Centric High Perform. Comput.*, 2019, pp. 50–57.

[46] D. Narayanan et al., "PipeDream: Generalized Pipeline Parallelism for DNN Training," in *Proc. 27th ACM Symp. Operating Syst. Princ.*, 2019, pp. 1–15.

[47] J. He et al., "FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2022, pp. 120–134.

[48] Z. Ma et al., "BaGuaLu: Targeting brain scale pretrained models with over 37 million cores," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2022, pp. 192–204.

[49] Z. Wang et al., "Merlin hugectr: GPU-accelerated recommender system training and inference," in *Proc. 16th ACM Conf. Recommender Syst.*, 2022, pp. 534–537.

[50] A. Shah, C.-Y. Wu, J. Mohan, V. Chidambaram, and P. Krähenbühl, "Memory optimization for deep networks," in *Proc. Int. Conf. Learn. Representations*, 2021.

[51] Z. Hu et al., "MegTaiChi: Dynamic tensor-based memory management optimization for DNN training," in *Proc. 36th ACM Int. Conf. Supercomputing*, 2022, pp. 25:1–25:13.

[52] A. Gruslys, R. Munos, I. Danihelka, M. Lanctot, and A. Graves, "Memory-efficient backpropagation through time," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 4125–4133.

[53] J. Liao et al., "Mimose: An input-aware checkpointing planner for efficient training on GPU," 2022, arXiv: *2209.02478*.

[54] M. Schuler, R. Membarth, and P. Slusallek, "XEngine: Optimal tensor rematerialization for neural networks in heterogeneous environments," *ACM Trans. Architecture Code Optim.*, vol. 20, 2022, Art. no. 17.
[55] J. Luo, J. Wu, and W. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5068–5076.
[56] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neurocomputing*, vol. 461, pp. 370–403, 2021.
[57] A. Sergeev and M. D. Balso, "Horovod: Fast and easy distributed deep learning in tensorflow," 2018, arXiv: *1802.05799*.
[58] S. H. Hashemi, S. A. Jyothi, and R. H. Campbell, "TicTac: Accelerating distributed deep learning with communication scheduling," in *Proc. 2nd SysML Conf.*, 2019, pp. 418–430.

Zan Zong received the PhD degree from the School of Software, Tsinghua University. He is a postdoctoral researcher with the Department of Computer Science and Technology, Tsinghua University, Beijing, China. His research interests include focused on high-performance deep learning systems and large-scale data processing systems.

Li Lin received the PhD degree from the School of Software, Tsinghua University. She is an assistant professor with the School of Computer Science and Engineering, Southeast University, Nanjing, China. Her research interests include focused on system performance modeling and sequence modeling (e.g., event sequence prediction and sequence generation).

Leilei Lin received the MS and PhD degrees from the School of Software from Yunnan University, Kunming, China. He was a postdoctoral scholar with Tsinghua University. Currently, he is a supervisor with School of Management, Capital Normal University. His research interests include data mining, process mining, service computing, and software engineering.

Lijie Wen received the BS and PhD degrees in computer science and technology from Tsinghua University, Beijing, China, in 2000 and 2007 respectively. He is currently an associate professor with the School of Software, Tsinghua University since 2012. His research interests include focused on process mining, process data management (e.g., log completeness, sequence clustering and prediction, process similarity, process indexing and retrieval), lifecycle management of computational workflow and natural language processing.

Yu Sun is an assistant professor with the Department of Computer Science, Nankai University, Tianjin, China. His current research interests include data quality, data cleaning and distributed computing systems. He has published papers in top conferences and journals including SIGMOD, SIGKDD, ICDE and *IEEE Transactions on Knowledge and Data Engineering*.